



FALL 2026

SEG3102 Software Design and Architecture

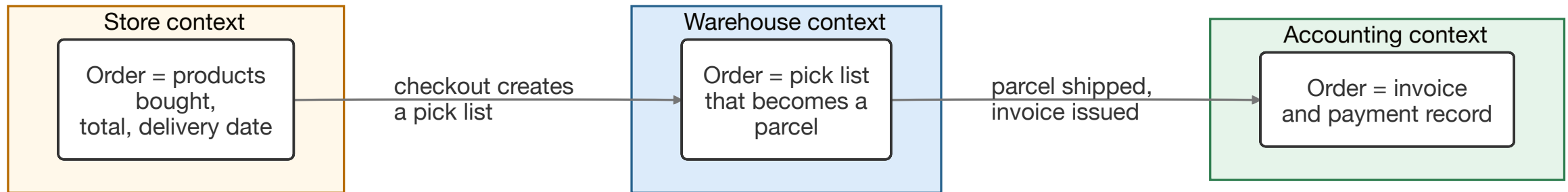
Domain-Driven Design: Behavior Design

Slide content used in this course is adapted from Prof. Stéphane Somé.

Last Lecture: Bounded context

Bounded context

Divide the ubiquitous language into several smaller languages, and assign each one to the explicit **context** in which it applies. That context is the **bounded context**.



Last lecture: Context diagram



Each arrow hides a question: who adapts to whom, and what happens when the other side changes? Those are the integration patterns. We will see them in later lectures.

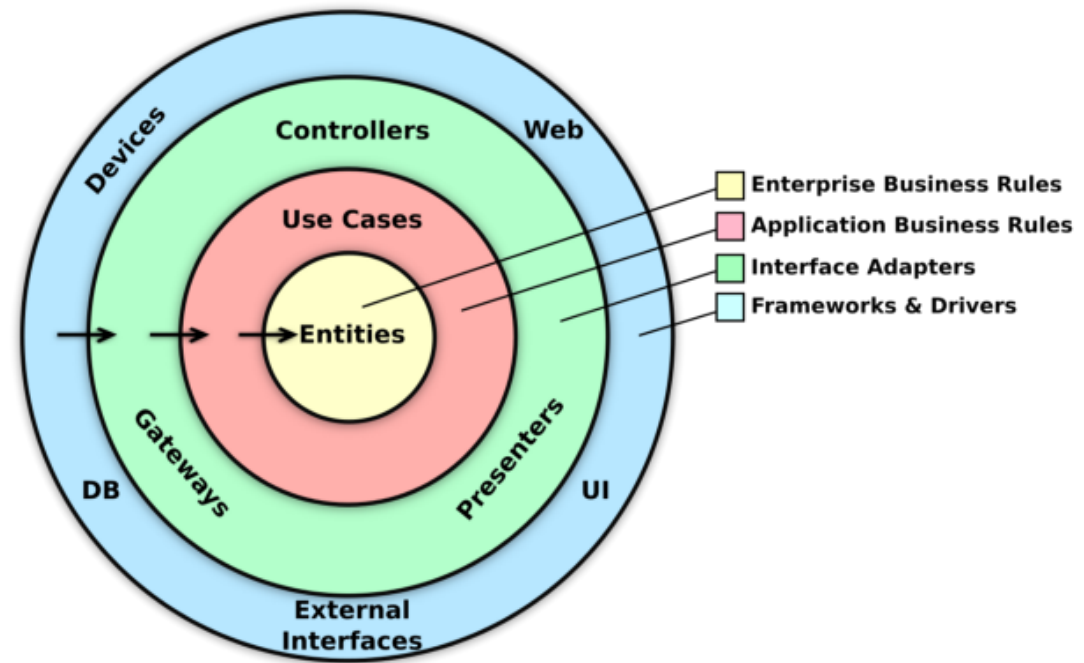
Last lecture: Modeling behavior



Last lecture: Application commands

- Command in application layer
 - triggers domain changes
- Identified from use case
 - points where request to system is performed

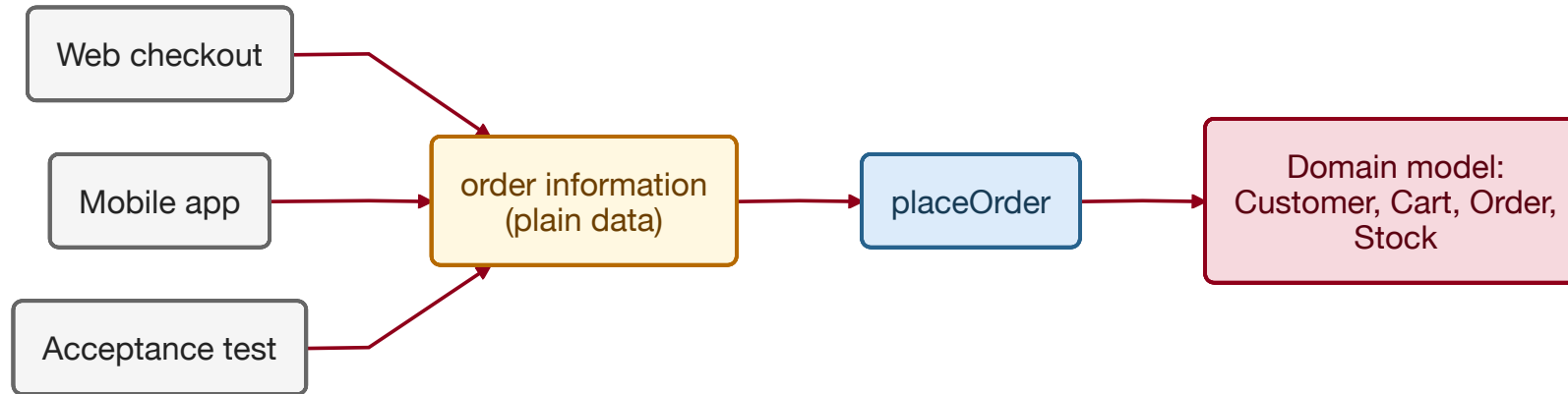
Application Command



Use Case component handles all commands from a use case

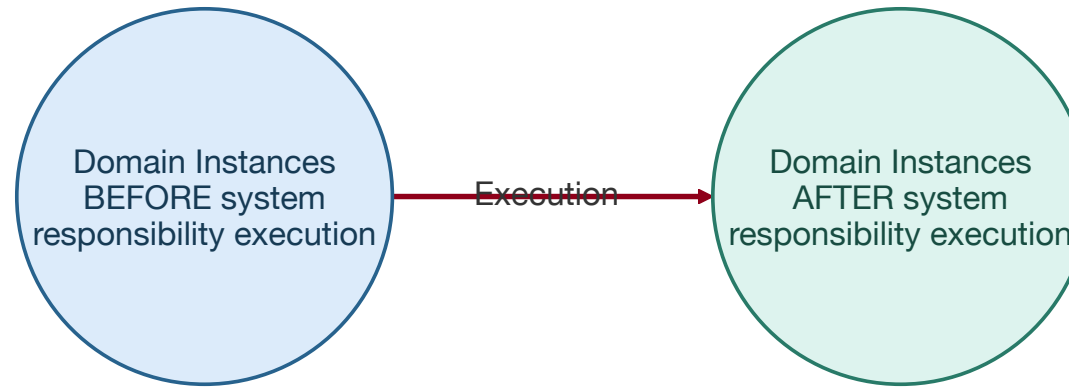
Last lecture: Data transfer objects (DTOs)

Application commands are independent of the user interface.



The command takes plain data, a **data transfer object**.

Last lecture: Contract



Preconditions: configuration required for execution

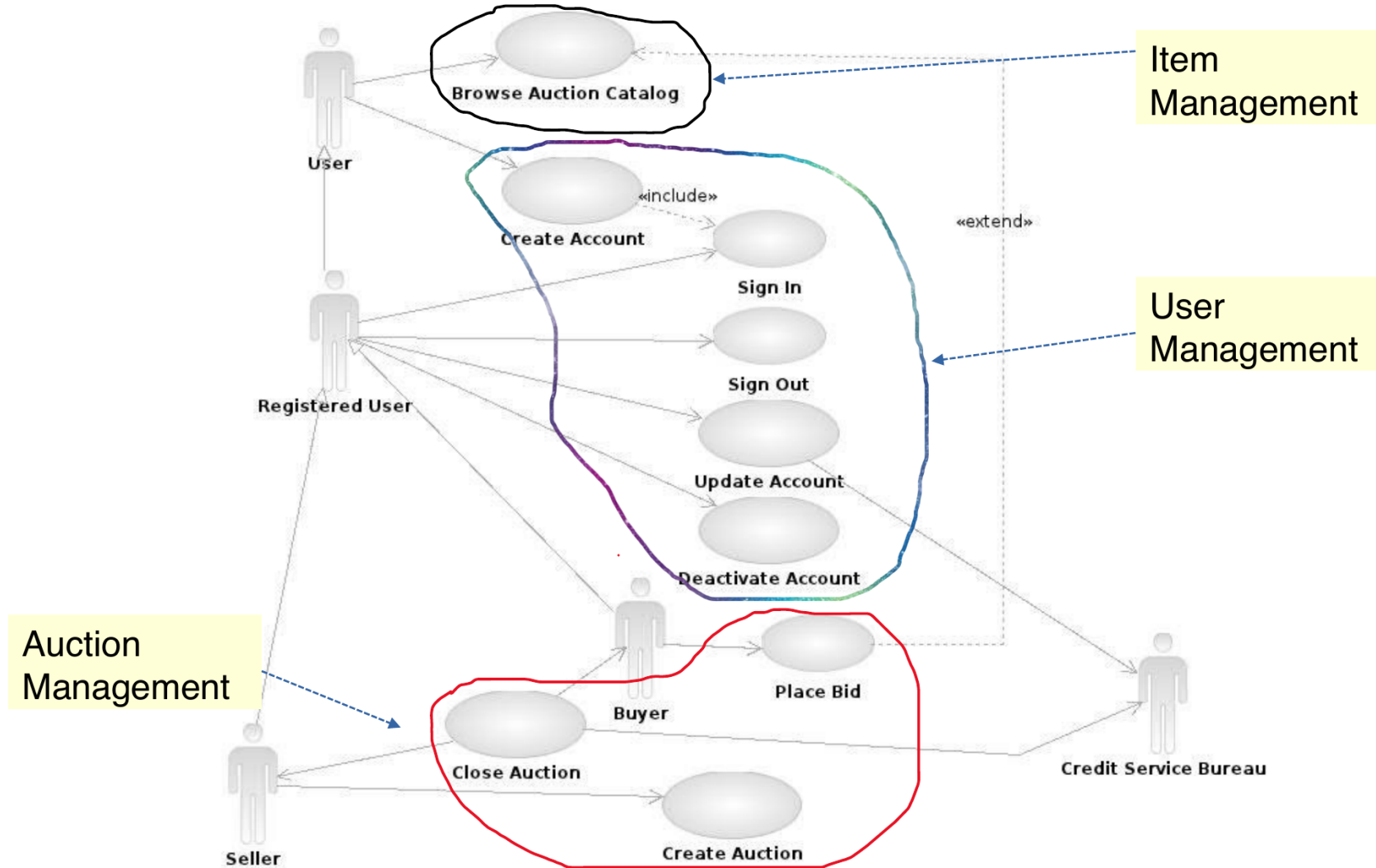
Postconditions: Differences between the configuration before and after

Outline

- Modeling behavior of the online auction system
- Behavior-driven development (BDD)

Modeling behavior of the online auction system

Sub-domains



Create Auction

*“The Create Auction use case allows a Seller to create an online auction by specifying auction information (including the start and end times) and, if it was not provided when the Seller created their user account, credit card information (see the Create Account use case). Once the auction has been created, Buyers can then place a bid on the item (see the Place Bid use case). To create an auction, **the Seller must be signed in (see the Sign In use case) and must not have any pending payments (see the Close Auction use case)**”*

Preconditions

Create Auction

“The Create Auction use case allows a Seller to create an online auction by specifying auction information (including the start and end times) and, if it was not provided when the Seller created their user account, credit card information (see the Create Account use case). Once the auction has been created, Buyers can then place a bid on the item (see the Place Bid use case). To create an auction, the Seller must be signed in (see the Sign In use case) and must not have any pending payments (see the Close Auction use case)”

Steps

Create Auction

*“The Create Auction use case allows a Seller to create an online auction by specifying auction information (including the start and end times) and, if it was not provided when the Seller created their user account, credit card information (see the Create Account use case). **Once the auction has been created, Buyers can then place a bid on the item** (see the Place Bid use case). To create an auction, the Seller must be signed in (see the Sign In use case) and must not have any pending payments (see the Close Auction use case)”*

Postcondition

Create Auction

Preconditions:

- Seller is signed in
- Seller doesn't have any pending payment

Steps

1. Seller selects to create an auction
2. System asks for auction information and optional credit card information
3. Seller submits auction information
4. System adds new auction to Seller's list of active auctions

Postcondition:

- New Auction added to Seller's account

Application command

2. System asks for auction information and optional credit card information
3. **Seller submits auction information**
4. System adds new auction to Seller's list of active auctions

Application command

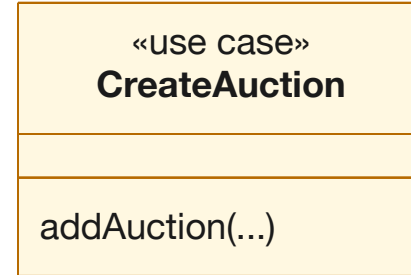
- Name should
 - be abstract, expressed at level of intent
 - independent from input medium, interface widget
 - ideally start with a verb (i.e. add, enter, end, make, ...)

Application command

2. System asks for auction information and optional credit card information
3. **Seller submits auction information**
4. System adds new auction to Seller's list of active auctions
 - What is the intent?
 - What is expected from the System?

Application command

- Good name: `addAuction`
 - reveals intention
 - starts with a verb
 - not linked to a device, input mode



Action Command

2. System asks for auction information and optional credit card information
3. Seller submits auction information
4. System adds new auction to Seller's list of active auctions

Auction Information: captures information about an auction, which includes the start time and duration of the auction, product information (title, description, image), starting price (minimum initial bid price), minimum bid increment, and auction category (the category in which the auction is listed)

Credit Card Information: Information about a credit card, which includes the credit card number, billing Information address, and the card's expiration date.

Data Transfer Object (DTO)

- Carries data between boundaries
 - simple data object
 - information hiding
 - avoids domain pollution
 - better modifiability & evolution

```
1 data class AuctionCreateDto(  
2     val startTime: LocalDateTime,  
3     val duration: Duration,  
4     val startPrice: BigDecimal,  
5     val minIncrement: BigDecimal,  
6     val seller: String,  
7     val category: String,  
8     var itemInfo: ItemCreateDto) {  
9     var creditCardInfo: CreditCardCreateDto? = null  
10 }
```

The application command in the code

application/usecases/CreateAuction.kt

```
1 package seg3x02.auctionsystem.application.usecases
2
3 import java.util.*
4
5 interface CreateAuction {
6     fun addAuction(auctionInfo: AuctionCreateDto): UUID?
7 }
```

The DTOs in the tutorial project

application/dtos/queries/AuctionCreateDto.kt

```

1 package seg3x02.auctionsystem.application.dtos.queries
2
3 import ...
4
5 data class AuctionCreateDto(val startTime: LocalDateTime,
6                             val duration: Duration,
7                             val startPrice: BigDecimal,
8                             val minIncrement: BigDecimal,
9                             val seller: String,
10                            val category: String,
11                            val itemInfo: ItemCreateDto) {
12     var creditCardInfo: CreditCardCreateDto? = null
13 }
```

application/dtos/queries/ItemCreateDto.kt

```

1 package seg3x02.auctionsystem.application.dtos.queries
2
3 data class ItemCreateDto(val title: String,
4                          val description: String) {
5     var image: ByteArray? = null
6 }
```

application/dtos/queries/AddressCreateDto.kt

```

1 package seg3x02.auctionsystem.application.dtos.queries
2
3 data class AddressCreateDto(val street: String,
4                             val city: String,
5                             val country: String,
6                             val postalCode: String)
```

application/dtos/queries/CreditCardCreateDto.kt

```

1 package seg3x02.auctionsystem.application.dtos.queries
2
3 import ...
4
5 data class CreditCardCreateDto(
6     val number: String,
7     val expirationMonth: Month,
8     val expirationYear: Year,
9     val accountFirstname: String,
10    val accountLastname: String,
11    val accountAddress: AddressCreateDto)
```

Contracts

- Specification of Command
 - Preconditions
 - Pre-existing state
 - Postconditions
 - State after execution

addAuction

2. System asks for auction information and optional credit card information

3. Seller submits auction information

4. **System adds new auction to Seller's list of active auctions**

- Creation of Auction
 - Setup based on arguments
 - Association to Seller
- Creation of Auction Item
 - Setup based on arguments
 - Association to Auction
- Creation of Credit Card (if information provided)
 - Setup based on arguments
 - Association to Seller
 - Association to Auction

addAuction

Preconditions:

- Seller is signed in
- Seller doesn't have any pending payment

Preexisting State:

- Current Seller
- Current Seller with no Pending Payment

Command: addAuction(seller, auctionInfo)

Preconditions:

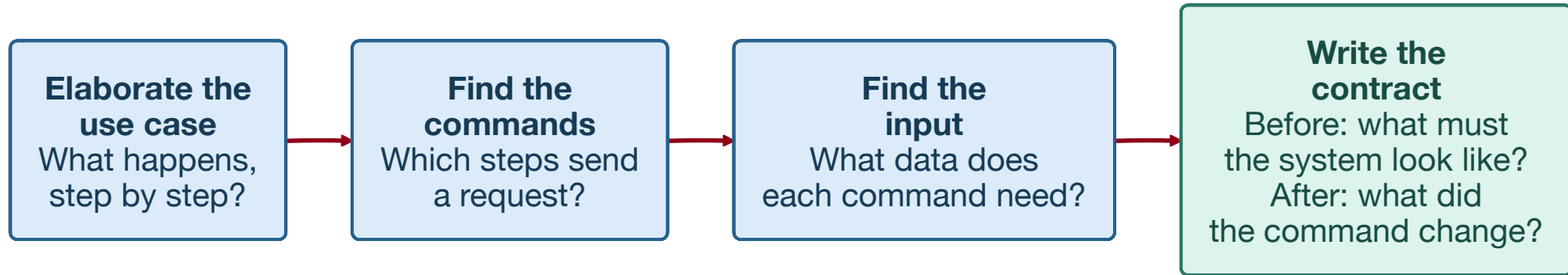
- signed-in **seller** (non null)
- non-existing Pending Payment for Seller

Post-conditions:

- a new Auction **auc** was created (creation of instance)
- **auc** was initialized from **auctionInfo** (modification of attributes)
- **auc** was added to **seller auctions** (creation of association)
- new Auction Item **aucIt** was created (creation of instance)
- **aucIt** was initialized from **auctionInfo** (modification of attributes)
- **aucIt** was set as **item** for **auc** (creation of association)
- **If** credit card infos were provided in **auctionInfo**
 - a new Credit Card **ccard** was created (creation of instance)
 - **ccard** was initialized from credit card infos (modification of attributes)
 - **ccard** was set as **seller's credit card** (creation of association)
 - **ccard** was set as **credit card** for **auc** (creation of association)

instance · **attribute** · **association**

Behavior design



What the contract gives us

The static model

Every noun is a candidate concept: Customer, Cart, Cart Line, Order, Order Line, Product, Stock. Every association created or destroyed is a candidate link between them.

**Structure modeling of the domain
(next lecture)**

The tests

Preconditions are the setup. The command is the action. Postconditions are what to check.

Behavior-Driven Development

Behavior-driven development and Gherkin

A tool to build the ubiquitous languages



The contract of `placeOrder` looks right. Your team has added a feature in the system for it. How do I know if it works?

Ehhh... We have some test cases... Let me find the test... it is called `testOrderFlow3`. It checks that no exception is thrown.



If we have 100 tests for the system, can we still find `testOrderFlow3`?

The domain expert cannot read `testOrderFlow3`.

We need something else

Test-driven development

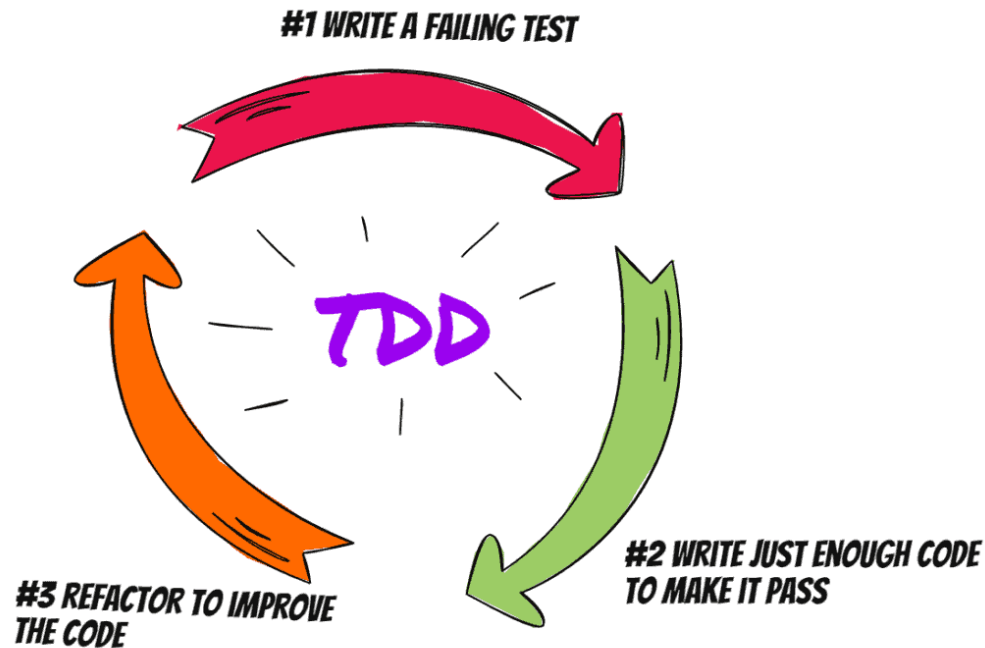


Image: Kristijan Kralj

Write the test first, then the code that makes it pass.

Which test do I write first? What is the easiest thing to test?

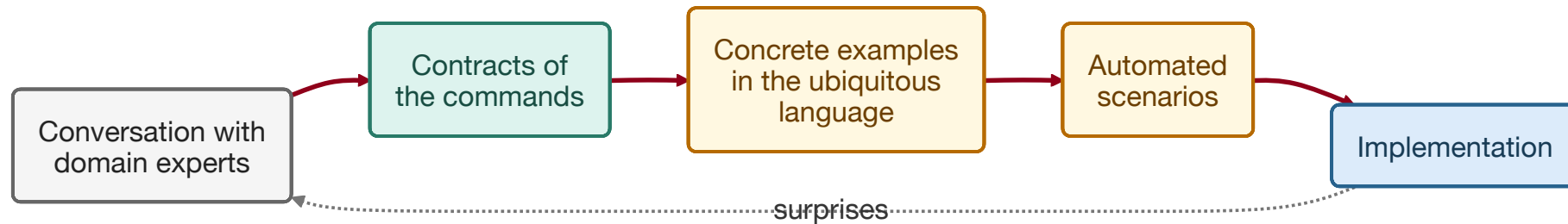


The system has to satisfy its use cases, so test their **behavior**: given this state, when this command runs, then this changes.

Behavior-driven development

Behavior-driven development (BDD)

Behavior-driven development is a practice where the team agrees on concrete **examples** of the system's behavior before building it, writes the examples in a shared language, and automates them as tests.

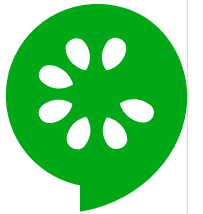


Examples are the shortest path to shared understanding

Gherkin: Given, When, Then

Gherkin is the language BDD tools use to write examples: cucumber.io/docs/gherkin

```
1 Feature: Place an order
2
3 Scenario: A customer orders items that are in stock
4   Given a signed-in customer with 2 "Wireless mouse" in the cart
5   And 10 "Wireless mouse" available in stock
6   When the customer places the order
7   Then a new order is added to the customer's orders
8   And 8 "Wireless mouse" remain available in stock
9   And the cart is empty
```



Given: the context, the state before

When: the action, one event

Then: the observable outcome

Examples that run as tests

✓ ✓ classpath:seg3x02.auctionsystem.contracts/close_auction.feature

Feature: Close auction at deadline

Scenario: Successful auction fee payment

- ✓ **Given** the auction is open
- ✓ **And** the auction deadline has passed
- ✓ **And** the auction seller credit card is able to settle the auction fee
- ✓ **When** the application command closeAuction is invoked
- ✓ **Then** the auction is marked as closed

Cucumber report of a Close Auction scenario in the auction project

test/.../contracts/steps/StepsDefinition.kt

```
1 Given("the auction is open") {
2     auction = createAuction(auctionRepository)
3     Assertions.assertThat(auction?.isclosed).isFalse()
4 }
5 When("the application command closeAuction is invoked") {
6     // ... build the CloseAuction use case
7     auction?.let { uc.closeAuction(it.id) }
8 }
9 Then("the auction is marked as closed") {
10     Assertions.assertThat(auction?.isclosed).isTrue()
11 }
```

A contract maps directly onto Gherkin

Contract

Preconditions:

- signed-in **customer** (non null)
- the **customer's cart** has at least one **cart line**
- for each **cart line**, the available stock covers the **quantity**

Command: `placeOrder(customer, orderInfo)`

Post-conditions:

- **ord** was added to the **customer's** orders
- the **available stock** of each product was reduced by the **ordered quantity**
- the **cart lines** were removed from the cart

Scenario

Given a signed-in customer

with 2 "Wireless mouse" in the cart

And 10 "Wireless mouse" available in stock

When the customer places the order

Then a new order is added to the customer's orders

And 8 "Wireless mouse" remain available in stock

And the cart is empty

A contract leads to many scenarios

The contract gives the main scenario. The stock precondition may not hold: the use case extension 4a gives a second scenario, where the order is not placed.

```
1 Feature: Place an order
2
3 Scenario: A customer orders items that are in stock
4   Given a signed-in customer with 2 "Wireless mouse" in the cart
5   And 10 "Wireless mouse" available in stock
6   When the customer places the order
7   Then a new order is added to the customer's orders
8   And 8 "Wireless mouse" remain available in stock
9   And the cart is empty
10
11 Scenario: An item in the cart is not available in the requested quantity
12   Given a signed-in customer with 2 "Wireless mouse" in the cart
13   And 1 "Wireless mouse" available in stock
14   When the customer places the order
15   Then the order is not placed
16   And 1 "Wireless mouse" remains available in stock
```

Scenario data embed domain rules

The domain expert gives us more cases for the stock rule:

```
1  Scenario: Plenty of stock
2    Given a signed-in customer with 2 "Wireless mouse" in the cart
3    And 10 "Wireless mouse" available in stock
4    When the customer places the order
5    Then the order is placed
6    And 8 "Wireless mouse" remain available in stock
7
8  Scenario: Exactly enough stock
9    Given a signed-in customer with 5 "Wireless mouse" in the cart
10   And 5 "Wireless mouse" available in stock
11   When the customer places the order
12   Then the order is placed
13   And 0 "Wireless mouse" remain available in stock
14
15 Scenario: Not enough stock
16   Given a signed-in customer with 6 "Wireless mouse" in the cart
17   And 5 "Wireless mouse" available in stock
18   When the customer places the order
19   Then the order is not placed
20   And 5 "Wireless mouse" remain available in stock
```

The **behavior** is identical in all three. Only the **data** changed. The rule is buried under copies of the same five lines.

Scenario outline: separate behavior from data

```

1  Scenario Outline: Ordering depends on the available stock
2    Given a signed-in customer with <in cart> <item> in the cart
3    And <available> <item> available in stock
4    When the customer places the order
5    Then the order is <outcome>
6    And <remaining> <item> remain available in stock

```

```

8  Examples:
9    | in cart | available | outcome   | remaining | item           |
10   | 2         | 10        | placed    | 8          | Wireless mouse |
11   | 5         | 5         | placed    | 0          | Wireless mouse |
12   | 6         | 5         | not placed| 5          | Wireless mouse |

```

- The outline states the behavior **once**
- The **Examples** table holds the data
- Each row runs as a separate test
- The expert can just add **rows**
- The columns come from the contract: quantity and available stock

The scenario uses the ubiquitous language

```
1 Scenario Outline: Ordering depends on the available stock
2   Given a signed-in customer with <in cart> <item> in the cart
3   And <available> <item> available in stock
4   When the customer places the order
5   Then the order is <outcome>
6   And <remaining> <item> remain available in stock
```

Every noun is a model concept:
Customer, Cart, Order, Stock.

Every verb is the command or a
change from the contract: *places the
order, remain available.*

Writing good scenarios

Avoid

```
1 Scenario: Checkout
2   Given I open the website
3   And I click "Cart"
4   And I click "Checkout"
5   And I fill in "12 Main Street"
6   And I click "Place order"
7   Then I see "Thank you"
```

Describes the UI, not the behavior.
Breaks whenever a button moves.

Prefer

```
1 Scenario: Order for in-stock items
2   Given a signed-in customer with 2 "Wireless mouse" in the cart
3   And 10 "Wireless mouse" available in stock
4   When the customer places the order
5   Then the order is placed
6   And 8 "Wireless mouse" remain available in stock
```

- Declarative: **what**, not how
- Concrete data: 2, 10, 8; or general description
- One behavior, in domain terms
- Runs as a test (Cucumber binds each step to code, you will learn more about this in the labs)

Summary

- **Behavior design:** elaborate the use case, find the commands and their input, write the contracts.
- **Behavior-driven development** turns contracts into examples that experts can read and tools can run.
- A **Gherkin scenario** is a contract as concrete examples in Given, When, Then.
- **Scenario outlines** state the behavior once and keep the data in a table.